HW0: Environment Setup and First C# Solution

Student: AHMED ALAA SAYED ABDALLAH

Student No. 230201920

Course: SENG 116 — Object Oriented Programming

Platform: Windows 10, .NET 10, Git, Visual Studio Code

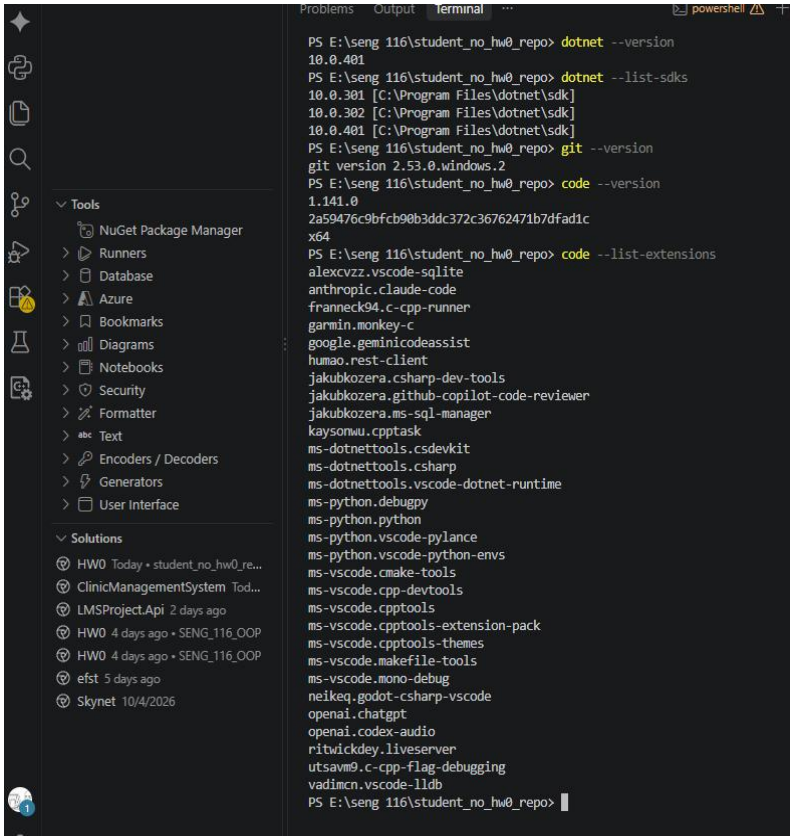
Repository : **https://github.com/ahmed-alaa236/SENG_116_OOP**

---

1. Objective

The objective of HW0 was to verify the development tools, create a C# solution containing a console application and an xUnit test project, implement the required number methods, observe normal and invalid-input behaviour, and record the work in Git.

Operating system shown by app



Method and Implementation

The solution contains two projects: HW0.App, which contains the console application, and HW0.Tests, which contains the automated tests. The solution file is HW0.slnx.

StudentCard

The `StudentCard` class represents a student's card. Its `Name` and `Number` properties store the state of the card. The constructor assigns these values when the object is created.

The `Summary()` method represents the behaviour of the class because it returns a formatted string containing the student's name and number.

The properties use `{ get; }`, so their values can be read but cannot be reassigned through those properties after construction. This helps keep the card's identity consistent.

DigitSum

The `DigitSum` method calculates the sum of the digits of a number. If the input is negative, the method first converts it to a positive value. It then uses a `while` loop to process the digits.

The expression `number % 10` obtains the last digit. That digit is added to the running sum, and integer division by 10 removes the last digit. The loop continues until the number becomes zero, after which the method returns the sum.

IsPrime

The `IsPrime` method returns `false` for every number smaller than 2. For other numbers, it checks possible divisors starting at 2 and ending at `n - 1`.

If `n % d == 0`, the number is divisible by `d` without a remainder, so the method returns `false`. If the loop finishes without finding a divisor, the method returns `true`.

CountPrimes

The `CountPrimes` method counts prime numbers from 2 up to and including the specified upper bound. It starts a counter at zero and uses a `for` loop to examine each number in that range.

For each value, it calls `IsPrime`. If the result is `true`, the counter is increased. Finally, the method returns the counter. Reusing `IsPrime` avoids duplicating the prime-checking logic.

4. Automated Testing

The provided `NumberToolsTests.cs` file contains 14 test cases:

-Four cases test `DigitSum`.

-Six cases test IsPrime.

-Four cases test CountPrimes.

The final test run completed successfully with the following result:

Total: 14 — Failed: 0 — Succeeded: 14 — Skipped: 0.

The solution also built successfully using dotnet build.

The handout predicts that before implementing IsPrime and CountPrimes, four DigitSum tests should pass and ten tests should fail because the other methods are placeholders. A separate screenshot or saved terminal output of that initial test run was not retained, so this is reported as the expected baseline from the handout rather than as an independently recorded result.

```
}
PS E:\seng 116\student_no_hw0_repo\SENG_116_OOP\HW0> dotnet build
Restore complete (0.6s)
  HW0.App net10.0 succeeded (0.2s) → HW0.App\bin\Debug\net10.0\HW0.App.dll
  HW0.Tests net10.0 succeeded (0.1s) → HW0.Tests\bin\Debug\net10.0\HW0.Tests.dll

Build succeeded in 1.2s
PS E:\seng 116\student_no_hw0_repo\SENG_116_OOP\HW0> dotnet test
Restore complete (0.7s)
  HW0.App net10.0 succeeded (0.2s) → HW0.App\bin\Debug\net10.0\HW0.App.dll
  HW0.Tests net10.0 succeeded (0.1s) → HW0.Tests\bin\Debug\net10.0\HW0.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 10.
0.12)
[xUnit.net 00:00:00.07]   Discovering: HW0.Tests
[xUnit.net 00:00:00.11]   Discovered:  HW0.Tests
[xUnit.net 00:00:00.12]   Starting:    HW0.Tests
[xUnit.net 00:00:00.19]   Finished:    HW0.Tests
  HW0.Tests test net10.0 succeeded (0.9s)

Test summary: total: 14, failed: 0, succeeded: 14, skipped: 0, duration: 0.9s
Build succeeded in 2.2s
PS E:\seng 116\student_no_hw0_repo\SENG_116_OOP\HW0> dotnet run --project .\HW0.App
\HW0.App.csproj
=------------ SENG 116 - HW0 ---------
.NET runtime : .NET 10.0.12
Operating sys. : Microsoft Windows 10.0.26200
Full name    : AHMED ALAA SAYED ABDALLAH
Student number : 230201920
Student: AHMED ALAA SAYED ABDALLAH (230201920)
Digit sum of the student number : 19
Is the digit sum prime? : True
Primes up to the digit sum : 8
PS E:\seng 116\student_no_hw0_repo\SENG_116_OOP\HW0>
```

---

5. Experiments

5.1 Experiment 1 — Normal Execution

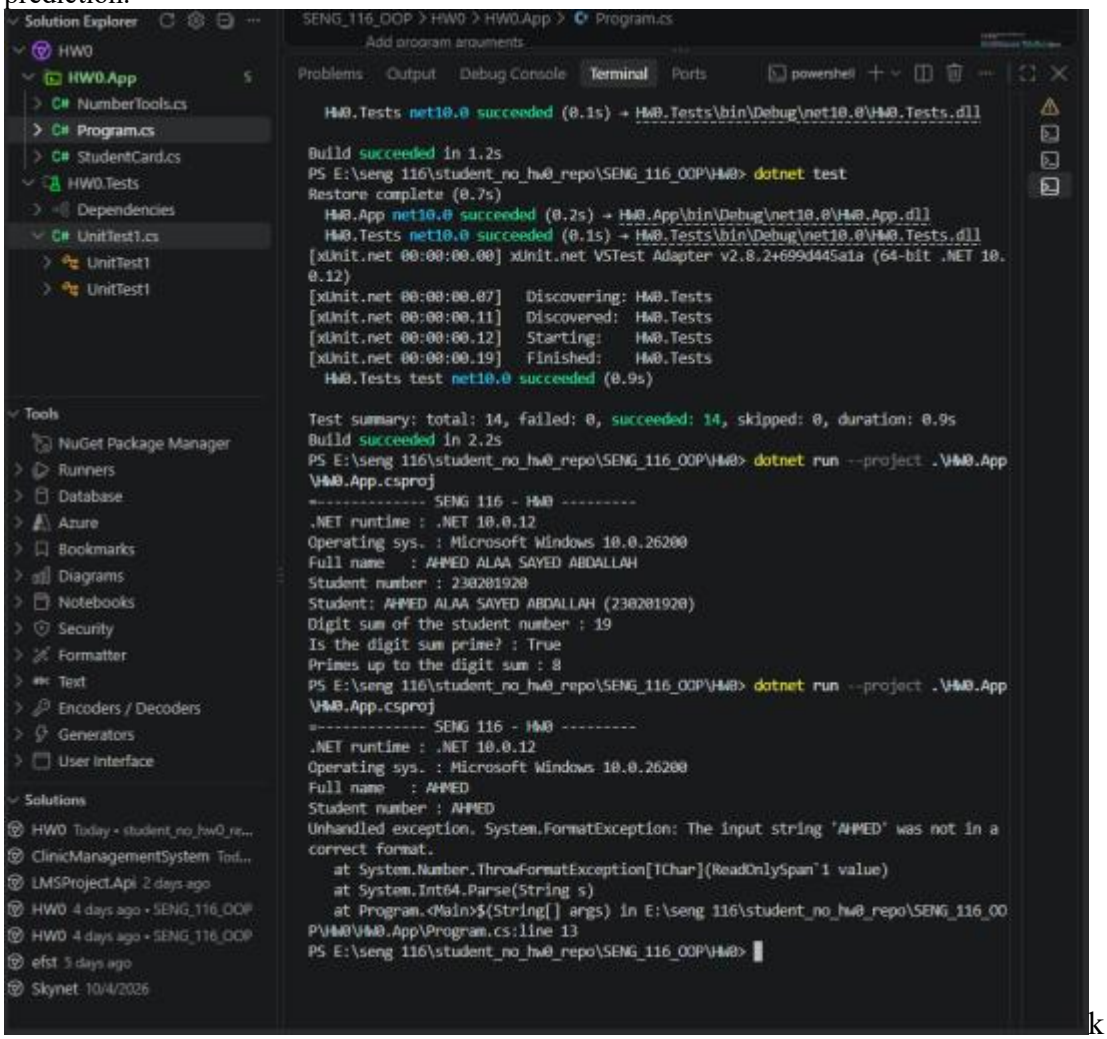
Predict: The digits of my student number, 230201920, sum to 19. Since 19 is a prime number, the program should report True for the prime check. There should be eight prime numbers from 2 through 19.

Run and Observe: The application displayed the student card summary, a digit sum of 19, a prime result of True, and a count of 8.

Explain: The digit sum is:

2 + 3 + 0 + 2 + 0 + 1 + 9 + 2 + 0 = 19.

The prime numbers up to and including 19 are 2, 3, 5, 7, 11, 13, 17, and 19. Therefore, the program's output matched the
prediction.



k

Build Output and Git

Predict: Building the application should generate compiled files and runtime metadata inside bin/Debug/net10.0. These generated files should not be tracked by Git because the .gitignore file excludes build output.

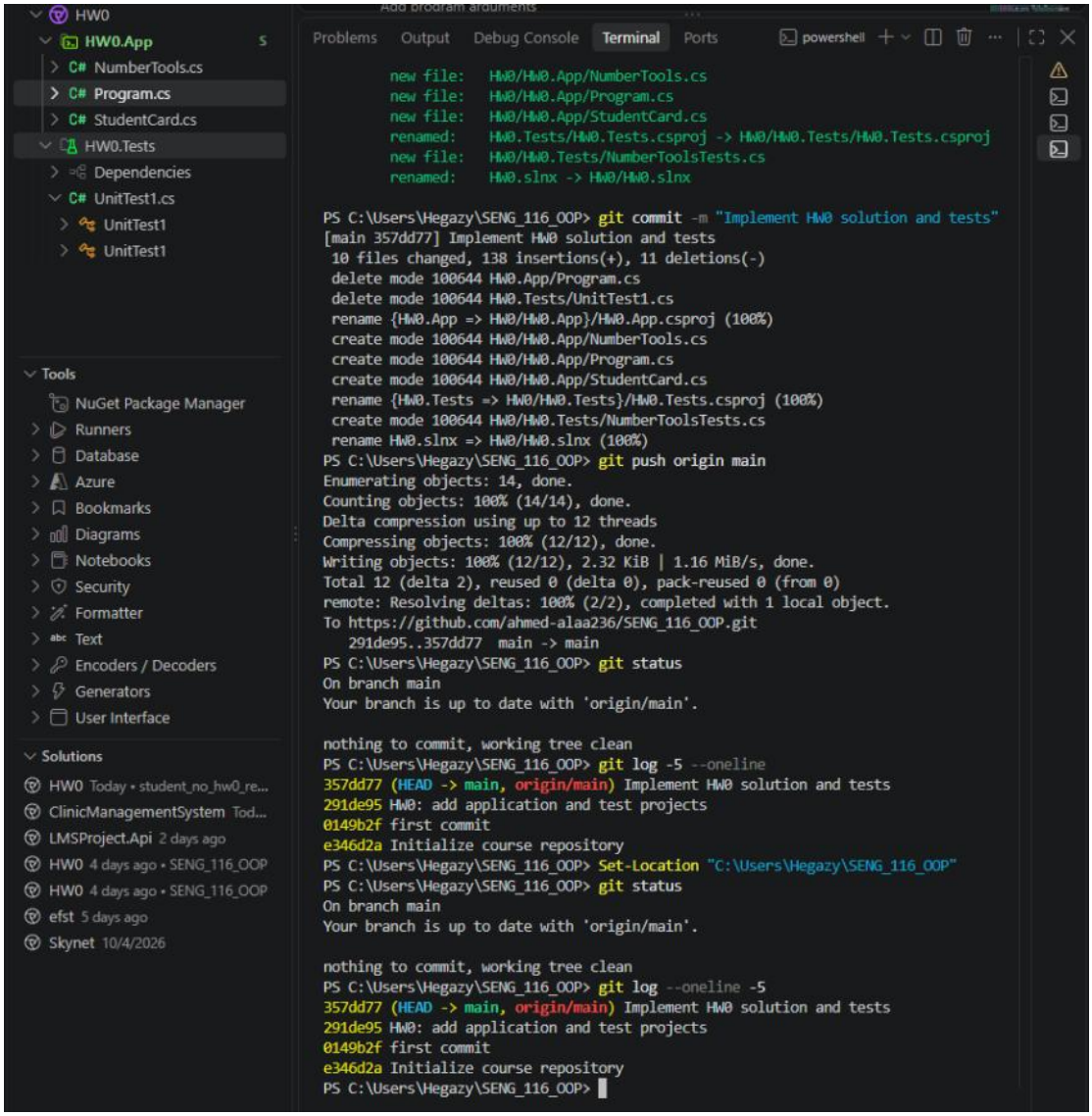
Run and Observe: The application output directory contained the following files:

- HW0.App.exe
- HW0.App.dll
- HW0.App.deps.json
- HW0.App.runtimeconfig.json
- HW0.App.pdb

The Git ignore checks in the repository confirmed that files under the application's bin/ directory and the project's obj/ directory were ignored. The repository later reported a clean working tree after the implementation was committed and pushed.

Explain: The .dll file contains the compiled application code. The .exe file is the Windows application host. The .deps.json file contains dependency information, while runtimeconfig.json contains runtime settings. The .pdb file contains debugging information.

These files are generated by the build process and can be regenerated from the source code d project files. Therefore, they do not need to be committed to Git.



--

7. Problems, conclusion, references and Git history Problems and resolution. The project existed in two different local folders. The E: copy contained the completed source and tests but was not itself a Git repository. The C: copy was linked to GitHub but initially contained the older project files. The source files were copied into the existing C: repository, the old test file was removed, and the solution was rebuilt and tested. The final commit was pushed successfully. Conclusion. The application builds successfully and all 14 supplied tests pass. The normal run produced the predicted results for student number 230201920. The invalid-input experiment demonstrated that parsing non-numeric text with long.Parse throws an unhandled FormatException. The repository is synced with origin/main and the working tree was clean at the time of the screenshot. References and assistance received. SENG 116 HW0 handout, “Environment Setup and Your First C# Solution,” version 4, especially Sections 8–15; SENG 116 Week 1 lecture notes; Microsoft .NET CLI output generated locally; xUnit test output generated locally; Git command output generated locally. ChatGPT was used to explain commands, troubleshoot repository structure, and help draft and organise this report. The student should review the report and ensure they can explain

the implementation. Appendix A — Git history The recorded output shows the following commits: 357dd77 Implement HW0 solution and tests 291de95 HW0: add application and test projects 0149b2f first commit e346d2a Initialize course repository Figure 4. Git push, clean working tree, and commit history.

8. 